

Gazebo 실습

World 작성 · Launch 구성 · 로봇 조작

cchyun@gmail.com

Part 1. World 파일 작성

1-1. 디렉토리 생성

- 패키지 내 `worlds` 디렉토리를 생성하여 World 파일을 관리

```
cd src/my_robot_description  
mkdir -p worlds
```

- Gazebo World는 `.world` 파일로 정의
 - `ground_plane` — 무한 평면 바닥
 - `sun` — 전역 광원

1-2. 바닥 평면 World

my_robot_description/worlds/empty_world.world :

```
<?xml version="1.0"?>
<sdf version="1.6">
  <world name="default">
    <physics type="ode">
      <max_step_size>0.001</max_step_size>
      <real_time_factor>1</real_time_factor>
    </physics>

    <include>
      <uri>model://ground_plane</uri>
    </include>

    <include>
      <uri>model://sun</uri>
    </include>
  </world>
</sdf>
```

- `ground_plane` 은 무한 평면 바닥, `sun` 은 전역 광원

Part 2. Launch 파일 작성

2-1. Gazebo 통합 Launch

- `my_robot_description/launch/gazebo.launch.py` :

```
import os
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import ExecuteProcess
from launch.substitutions import Command
from launch_ros.actions import Node

def generate_launch_description():
    pkg_dir = get_package_share_directory('my_robot_description')
    xacro_file = os.path.join(pkg_dir, 'urdf', 'turtlebot.xacro')
    world_file = os.path.join(pkg_dir, 'worlds', 'empty_world.world')
    rviz_file = os.path.join(pkg_dir, 'rviz', 'turtlebot_gazebo.rviz')

    robot_description = Command(['xacro ', xacro_file])

    rviz_args = ['-d', rviz_file] if os.path.exists(rviz_file) else []

    return LaunchDescription([
        ExecuteProcess(
            cmd=['gazebo', '--verbose', world_file,
                '-s', 'libgazebo_ros_init.so',
                '-s', 'libgazebo_ros_factory.so'],
            output='screen'
        ),
        Node(
            package='robot_state_publisher',
            executable='robot_state_publisher',
            parameters=[{'robot_description': robot_description}]
        ),
        Node(
            package='gazebo_ros',
            executable='spawn_entity.py',
            arguments=['-topic', 'robot_description',
                '-entity', 'turtlebot', '-z', '0.3'],
            output='screen'
        ),
        Node(
            package='rviz2',
            executable='rviz2',
            arguments=rviz_args,
        ),
    ])

```

2-2. CMakeLists.txt 수정

- `worlds` 디렉토리로 설치에 포함시켜야 함

```
install(  
  DIRECTORY urdf launch rviz worlds  
  DESTINATION share/${PROJECT_NAME}  
)
```

2-3. 빌드

```
cd ~/Workspace/ros2-ws  
colcon build --packages-select my_robot_description  
source install/setup.bash
```


Part 3. Gazebo 실행 및 로봇 조작

3-1. Launch 실행

```
ros2 launch my_robot_description gazebo.launch.py
```

- Gazebo 창이 열리고, 바닥 평면 위에 로봇이 생성.
- 본체, 바퀴, LiDAR, 카메라가 보임.

3-2. 토픽 확인

```
# 터미널 2
ros2 topic list
```

실행 중인 주요 토픽:

토픽	설명
/cmd_vel	속도 명령 (입력)
/odom	주행 기록 (출력)
/scan	LiDAR 데이터 (출력)
/camera/image_raw	카메라 영상 (출력)
/tf	TF 트리
/tf_static	정적 TF

3-3. 키보드 조작

```
# 터미널 2
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

키	동작
i	전진
,	후진
j	좌회전 (제자리)
l	우회전 (제자리)
u	좌전진
o	우전진
k	정지
q/z	최고 속도 증가/감소

Gazebo 창에서 로봇이 이동하는 것을 확인하세요.

Part 4. 장애물이 있는 World (심화)

4-1. World 파일 작성

- `my_robot_description/worlds/obstacle_world.world` :

```
<?xml version="1.0"?>
<sdf version="1.6">
  <world name="obstacle_world">
    <physics type="ode">
      <max_step_size>0.001</max_step_size>
      <real_time_factor>1</real_time_factor>
    </physics>

    <include><uri>model://ground_plane</uri></include>
    <include><uri>model://sun</uri></include>

    <!-- Obstacle: box -->
    <model name="box1">
      <pose>1.0 0.0 0.25 0 0 0</pose>
      <static>true</static>
      <link name="link">
        <collision name="collision">
          <geometry>
            <box><size>0.5 0.5 0.5</size></box>
          </geometry>
        </collision>
        <visual name="visual">
          <geometry>
            <box><size>0.5 0.5 0.5</size></box>
          </geometry>
          <material>
            <ambient>1 0 0 1</ambient>
          </material>
        </visual>
      </link>
    </model>
```

4-1. World 파일 작성

```
<!-- Obstacle: cylinder -->
<model name="cylinder1">
  <pose>0.5 1.0 0.25 0 0 0</pose>
  <static>true</static>
  <link name="link">
    <collision name="collision">
      <geometry>
        <cylinder><radius>0.2</radius><length>0.5</length></cylinder>
      </geometry>
    </collision>
    <visual name="visual">
      <geometry>
        <cylinder><radius>0.2</radius><length>0.5</length></cylinder>
      </geometry>
      <material>
        <ambient>0 1 0 1</ambient>
      </material>
    </visual>
  </link>
</model>
</world>
</sdf>
```

4-2. 실행

- launch 파일의 `world_file` 경로를 `obstacle_world.world`로 변경 후 실행

```
world_file = os.path.join(pkg_dir, 'worlds', 'obstacle_world.world')
```

- 변경 후 빌드 및 실행

```
cd ~/Workspace/ros2-ws  
colcon build --packages-select my_robot_description  
source install/setup.bash  
ros2 launch my_robot_description gazebo.launch.py
```


Part 5. 심화 미션



- LiDAR 데이터를 구독하여 장애물 감지 시 자동 회피하는 Python 노드 작성
 - 구현 목표: `/scan` 구독 → 전방 0.5m 이내 장애물 감지 시 회전